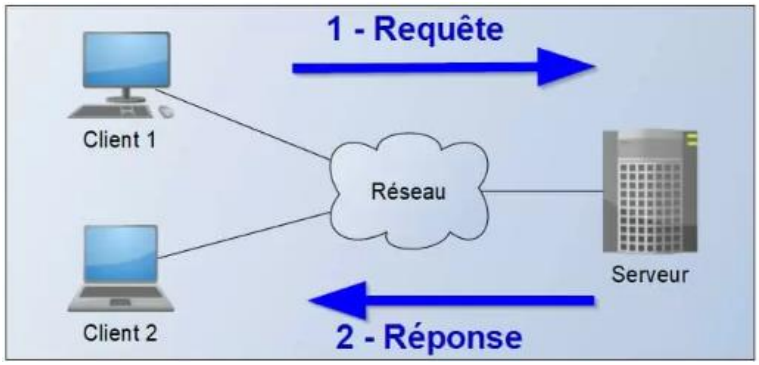
LES ACCES DISTANTS SÉCURISÉS

I - Introduction

Les accès distants aux services de l'entreprise correspondent aux possibilités d'accéder à des infrastructures même en dehors des locaux. Par exemple, la prise de contrôle à distance d'un ordinateur pour un dépannage informatique.

D'une façon générale, un accès distant repose sur un échange de type client/serveur :



Cela nécessite 4 composants :

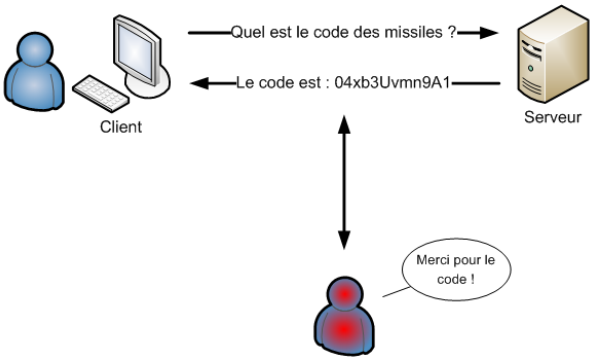
- Un protocole
- Un réseau
- Un service côté serveur
- Un logiciel côté client

Chaque protocole définit un ensemble de règles permettant au client d'émettre une requête vers le serveur, et au serveur de retourner une réponse au client.

II - Le protocole Telnet *(port tcp/23)*

Il s'agit d'un protocole très simple créé dans les années 1980. Il permet de prendre le contrôle à distance d'un ordinateur. Son principal inconvénient est que les données échangées sont transférées en clair sur le réseau. Il n'y a aucun chiffrement.

Voici ce qui pourrait se passer si un PC militaire demande à un serveur de l'armée le code de lancement de missiles, comme sur la figure suivante.



source : https://openclassrooms.com/fr/

III - Le protocole SSH *(port tcp/22)*

La théorie de la cryptographie asymétrique

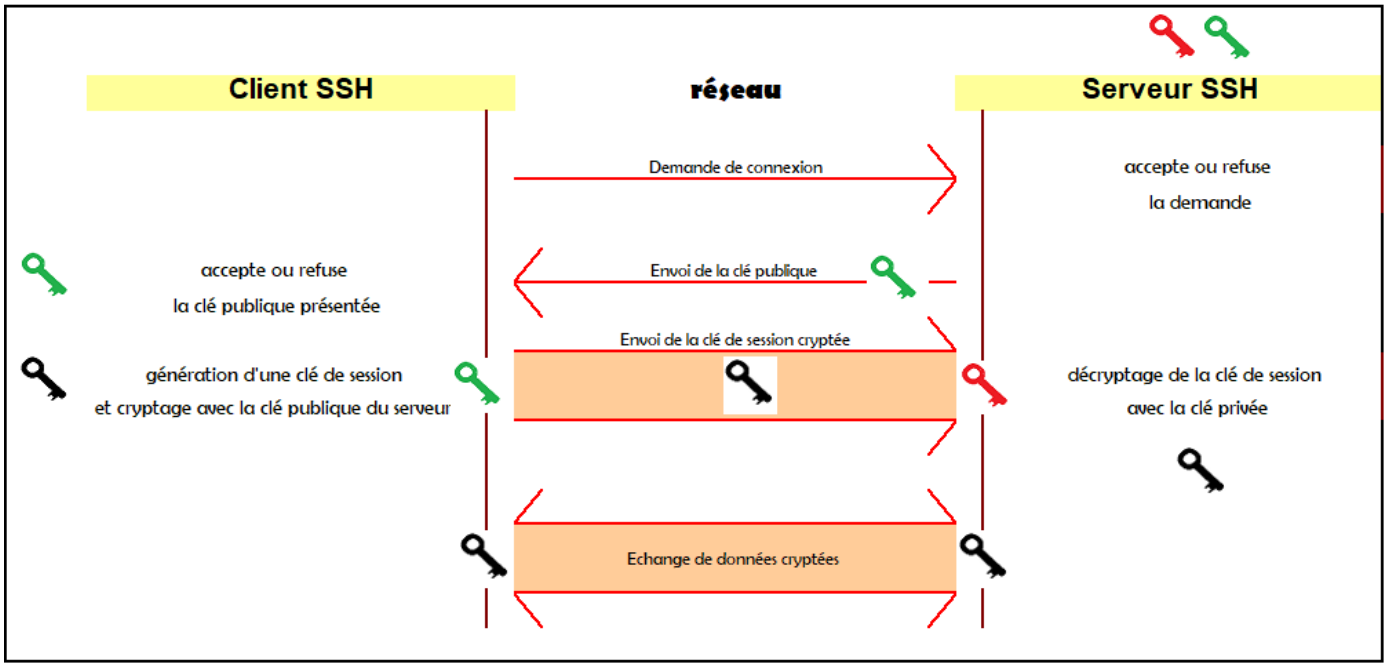
SSH utilise la cryptographie asymétrique RSA ou DSA. En cryptographie asymétrique, chaque entité dispose d'un couple de clef : une clé publique et une clef privée. La clé publique peut être librement publiée tandis que la clef privée doit rester secrète. La connaissance de la clef publique ne permet pas d'en déduire la clé privée.

Si Alice veut envoyer un message confidentiel à Bob, elle doit le chiffrer avec la clef publique de Bob et lui envoyer sur un canal qui n'est pas forcément sécurisé. Seul Bob pourra déchiffrer ce message en utilisant sa clef privée.

La théorie de la cryptographie symétrique

SSH utilise également la cryptographie symétrique. Son principe est simple : si Alice veut envoyer un message confidentiel à Bob, Alice et Bob doivent d'abord posséder une même clef secrète. Alice chiffre le message avec la clé secrète puis l'envoie à Bob sur un canal qui n'est pas forcément sécurisé. Bob déchiffre alors le message grâce à la clef secrète. Toute autre personne en possession de la clef secrète peut également déchiffrer le message.

Les algorithmes de chiffrement symétrique sont beaucoup moins gourmands en ressources processeur que ceux de chiffrement asymétrique… mais le gros problème est l'échange de la clef secrète entre Alice et Bob. Dans le protocole SSH, la cryptographie asymétrique est utilisée au début de la communication pour que Alice et Bob puissent s'échanger une clef secrète de manière sécurisée, puis la suite la communication est sécurisée grâce à la cryptographie symétrique en utilisant la clef secrète ainsi échangée.



L'établissement d'une connexion SSH

Un serveur SSH dispose d'un couple de clefs RSA stocké dans le répertoire /etc/ssh/ et généré lors de l'installation du serveur. Le fichier ssh_host_rsa_key contient la clef privée et a les permissions 600. Le fichier ssh_host_rsa_key.pub contient la clef publique et a les permissions 644.

Voici les étapes effectuées lors de l'établissement d'une connexion SSH :

1. Le serveur envoie sa clef publique au client. Celui-ci vérifie qu'il s'agit bien de la clef du serveur, s'il l'a déjà reçue lors d'une connexion précédente.
2. Le client génère une clef secrète et l'envoie au serveur, en chiffrant l'échange avec la clef publique du serveur *(chiffrement asymétrique)*. Le serveur déchiffre cette clef secrète en utilisant sa clé privée, ce qui prouve qu'il est bien le vrai serveur.
3. Pour le prouver au client, il chiffre un message standard avec la clef secrète et l'envoie au client. Si le client retrouve le message standard en utilisant la clef secrète, il a la preuve que le serveur est bien le vrai serveur.
4. Une fois la clef secrète échangée, le client et le serveur peuvent alors établir un canal sécurisé grâce à la clef secrète commune *(chiffrement symétrique)*.
5. Une fois que le canal sécurisé est en place, le client va pouvoir envoyer au serveur le login et le mot de passe de l'utilisateur pour vérification. Le canal sécurisé reste en place jusqu'à ce que l'utilisateur se déconnecte.

La seule contrainte est de s'assurer que la clef publique présentée par le serveur est bien sa clef publique sinon le client risque de se connecter à un faux serveur qui aurait pris l'adresse IP du vrai serveur. Une bonne méthode est par exemple de demander à l'administrateur du serveur quel est le fingerprint de la clef publique du serveur avant de s'y connecter pour la première fois. Le fingerprint d'une clef publique est une chaîne de 32 caractères hexadécimaux à peu près unique pour chaque clef (un hachage) ; il s'obtient grâce à la commande `ssh-keygen -if /etc/ssh/ssh_host_rsa_key.pub`.

Authentification par clé publique

[C] Génération d'une paire de clés (K,k)
[C->S] Envoi du login et de la clé publique K
[S] Recherche de la clé publique K de l'utilisateur dans ~user/.ssh/authorized_keys.
[S] Génération d' un nombre aléatoire A, et calcul d' un challenge C, résultat du chiffrement de A par la clé publique K de l'utilisateur
[C<-S] Envoi du challenge C
[C] Calcule de R, résultat du déchiffrement du challenge C par la clé privée k de l'utilisateur suivi du hachage MD5
[C->S] Envoi de la réponse R
[S] Si R == MD5(A), confirmer l'authentification.

… la session peut être établie.

IV - Le protocole TLS/SSL *(port tcp/443)*

Le protocole TLS (ou SSL) fonctionne suivant un mode client-serveur. Il permet de satisfaire les objectifs de sécurité suivants :

- l'authentification du serveur ;
- la confidentialité des données échangées (ou session chiffrée) ;
- l'intégrité des données échangées ;
- de manière optionnelle, l'authentification du client (mais dans la réalité celle-ci est souvent assurée par la couche applicative).

Le protocole est très largement utilisé, et sa mise en œuvre est facilitée par le fait que les protocoles de la couche application (comme HTTP) n'ont pas à être profondément modifiés pour utiliser une connexion sécurisée. De ce fait, HTTP associé à TLS/SSL a donné naissance au protocole HTTPS.